

Web Scraping with Python

Automated
Data
Extraction
& Pipeline

Presenter:

Venugopalakrishna
Vanka

Introduction to Web Scraping

- **Definition:** Web scraping refers to the automated process of extracting structured information from websites using custom scripts or applications.
- **Data Handling:** Allows developers to collect raw HTML, parse relevant fields, and format it for presentation or storage.
- **Integration:** Serves as a critical data collection layer for downstream analytics, machine learning pipelines, and monitoring tools.

Why Web Scraping?

- **Time Efficiency:** Automates data collection, reducing execution time from hours of manual copying to seconds.
- **Accuracy & Uniformity:** Eliminates human error and outputs standardized data formats consistently.
- **System Integration:** Plugs directly into databases and automation pipelines for real-time ingestion.
- **API Alternative:** Unlocks valuable web data even when official public APIs are unavailable or rate-limited.

Where It Is Used

- **Search Engine Indexing:** Engines like Google and DuckDuckGo scrape pages to index web content.
- **Brand & PR Monitoring:** Marketing teams track brand mentions, reviews, and sentiment online.
- **Trend Analysis:** Real-time monitoring of breaking news, social trends, and market shifts.
- **Competitive Intelligence:** Scraping competitor products, pricing strategies, and catalog updates.

Tooling & Library Overview

TOOL CATEGORY	LIBRARIES	KEY FUNCTION & USE CASE
HTTP Clients	Requests, Urllib	Fast static HTML fetching. Directly requests web content without browser overhead.
Browser Automation	Selenium	Automates full browsers. Essential for JavaScript-heavy, dynamic single-page applications.
HTML Parsers	lxml, BeautifulSoup	Navigates and parses HTML trees to extract text via XPath or CSS selectors.

Web Scraping Pipeline

1. Download

Fetch raw HTML content from target URLs using `Requests` or `Selenium`.



2. Parse

Analyze document tree and extract target elements using `lxml` XPath selectors.



3. Store

Write clean data locally into structured formats like CSV, JSON, or database tables.

Implementation: Downloading & Parsing

```
import requests
from lxml import html

url = 'https://www.w3schools.com/html/html_tables.asp'

if __name__ == '__main__':
    req = requests.get(url, verify=False)
    result = []

    if req.status_code == 200:
        doc = html.fromstring(req.content)
        table_elem = doc.xpath('//*[table[@id="customers"]'])[0]

        for tr_elem in table_elem.xpath('//*[tr]'):
            row = []
            tds = tr_elem.xpath('//*[td]') # Define tds before loop
            for td_elem in tds:
                row.append(td_elem.text_content().strip())
            if row:
                result.append(row)
```

Implementation: Storing Data to CSV

```
import csv

# Step 3: Store retrieved data safely in CSV format
if result:
    output_file = 'scraped_data.csv'

    # Using context manager auto-closes file and handles formatting
    with open(output_file, 'w', newline='', encoding='utf-8') as f:
        writer = csv.writer(f)
        writer.writerows(result) # Bulk write rows

    print("Data successfully exported to scraped_data.csv")
else:
    print("No data retrieved or request failed.")
```


Conclusion & Best Practices

- **Small-Scale Scraping:** Using `Requests` + `lxml` provides lightweight, fast automation for static web pages.
- **Large-Scale Scraping:** For enterprise data extraction, transition to robust frameworks like **Scrapy** (async concurrency, proxies, retry middleware).
- **Ethical Guidelines:** Respect target site terms of service, observe `robots.txt`, and enforce request rate limiting to avoid overloading servers.

References: Python Official Documentation (python.org) | W3Schools HTML Specs